

Lab 4 VibeCoding: Chat History

Student: Kundhan Vudutha (PES1UG24AM327)
Assigned repo: SETAPESU26/27_centipede
LLM used: Claude (claude.ai)
Date: 6 October 2026
Excerpt: the prompt used to complete the four tasks and Claude's response.

Prompt

I'm doing a lab where I use an AI to fix and extend code. Repo: https://github.com/SETAPESU26/27_centipede It's a single-file Pygame Centipede clone (game.py). The README lists four tasks:

1. Fix the bug in hit_mushroom so a mushroom survives MUSHROOM_HP (4) hits instead of 3.
2. Implement mushroom_color(hp) so a mushroom visibly flashes white when it's hit.
3. Implement on_segment_hit(segment, score) to show a spark burst where the segment was shot.
4. Implement wave_speed_bonus(wave) to return $1 + 0.15 * (\text{wave} - 1)$.

Constraints: only edit game.py, keep every existing call site working, no new dependencies.

Do the tasks one at a time. For each: explain the cause or design in 2 to 3 lines, show a unified diff, and give a one-line commit message. Then verify all four with a short headless test (SDL_VIDEODRIVER=dummy) and show the output. End with the complete final game.py.

Response

Task 1: Fix mushroom durability

hit_mushroom subtracts one HP and then deletes the mushroom when HP is ≤ 1 , so a 4 HP mushroom is removed on its third hit. Changing the check to ≤ 0 removes it only after all four hits.

```
diff --git a/game.py b/game.py
index b6b6e0e..a491eb1 100644
--- a/game.py
+++ b/game.py
@@ -70,7 +70,7 @@ class Game:

     def hit_mushroom(self, cell):
         self.mushrooms[cell] -= 1
-        if self.mushrooms[cell] <= 1:
+        if self.mushrooms[cell] <= 0:
             del self.mushrooms[cell]
             self.score += 5
```

Commit: Task 1: fix mushroom durability (clear at 0 HP, not 1)

Task 2: mushroom_color flashes white on hit

mushroom_color(hp) only gets HP, which can't tell which mushroom was just shot. I added an optional flashing argument (default False, so the old call form still works) and a per-cell self.flash timer set in hit_mushroom and counted down in update. The flash lasts 0.1 s because a single frame at 60 fps (16 ms) is too short to see; otherwise it returns None so the default HP fade still applies.

```
diff --git a/game.py b/game.py
index a491eb1..13085bd 100644
--- a/game.py
+++ b/game.py
@@ -6,11 +6,14 @@ WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4
+FLASH_TIME = 0.1

-def mushroom_color(hp):
+def mushroom_color(hp, flashing=False):
+    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""
-    pass
+    if flashing:
+        return (255, 255, 255)
+    return None

def on_segment_hit(segment, score):
@@ -48,6 +51,7 @@ class Game:
    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
+        self.flash = {}
+        for _ in range(45):
+            self.mushrooms[(random.randint(1, ZONE_TOP - 2), random.randint(0, COLS - 1))] = MUSHROOM_HP
```

```

        self.respawn()
@@ -72,7 +76,10 @@ class Game:
    self.mushrooms[cell] -= 1
    if self.mushrooms[cell] <= 0:
        del self.mushrooms[cell]
+       self.flash.pop(cell, None)
+       self.score += 5
+
    else:
+       self.flash[cell] = FLASH_TIME

    def split_chain(self, chain, index):
        segment = chain[index]
@@ -105,6 +112,7 @@ class Game:
    if self.state != "play":
        return
    self.invulnerable = max(0.0, self.invulnerable - dt)
+   self.flash = {cell: t - dt for cell, t in self.flash.items() if t - dt > 0}
    self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * PLAYER_SPEED * dt
    self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * PLAYER_SPEED * dt
    self.x = max(10, min(WIDTH - 10, self.x))
@@ -132,7 +140,7 @@ class Game:
    def draw(self, screen):
        screen.fill((8, 8, 16))
        for (row, col), hp in self.mushrooms.items():
-           color = mushroom_color(hp) or (200 - (MUSHROOM_HP - hp) * 40, 80, 170)
+           color = mushroom_color(hp, (row, col) in self.flash) or (200 - (MUSHROOM_HP - hp) * 40, 80, 170)
            center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
            pygame.draw.circle(screen, color, center, CELL // 2 - 1)
            pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3, center[1], 6, CELL // 2 - 1))

```

Commit: Task 2: mushroom_color flashes a mushroom white when it is hit

Task 3: on_segment_hit spark burst

The return value of on_segment_hit is ignored, so it adds 12 particles to a module-level sparks list at the segment's centre, each with a random direction and speed. update moves and ages them, draw renders them fading from yellow to orange while shrinking, and reset clears them.

```

diff --git a/game.py b/game.py
index 13085bd..c3b09cc 100644
--- a/game.py
+++ b/game.py
@@ -7,6 +7,8 @@ ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4
FLASH_TIME = 0.1
+SPARK_LIFE = 0.4
+sparks = []

def mushroom_color(hp, flashing=False):
@@ -18,7 +20,11 @@ def mushroom_color(hp, flashing=False):

def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""
-   pass
+   cx = segment.col * CELL + CELL / 2
+   cy = segment.row * CELL + CELL / 2
+   for _ in range(12):
+       velocity = pygame.Vector2(random.uniform(80, 220), 0).rotate(random.uniform(0, 360))
+       sparks.append([pygame.Vector2(cx, cy), velocity, SPARK_LIFE])

def wave_speed_bonus(wave):
@@ -52,6 +58,7 @@ class Game:
    self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
    self.mushrooms = {}
    self.flash = {}
+   sparks.clear()
    for _ in range(45):
        self.mushrooms[(random.randint(1, ZONE_TOP - 2), random.randint(0, COLS - 1))] = MUSHROOM_HP
    self.respawn()
@@ -113,6 +120,10 @@ class Game:
    return
    self.invulnerable = max(0.0, self.invulnerable - dt)
    self.flash = {cell: t - dt for cell, t in self.flash.items() if t - dt > 0}
+   for spark in sparks:
+       spark[0] += spark[1] * dt
+       spark[2] -= dt
+   sparks[:] = [spark for spark in sparks if spark[2] > 0]
    self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * PLAYER_SPEED * dt
    self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * PLAYER_SPEED * dt
    self.x = max(10, min(WIDTH - 10, self.x))
@@ -148,6 +159,9 @@ class Game:
    for index, segment in enumerate(chain):
        center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
        pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL // 2)
+   for pos, _, life in sparks:
+       fade = life / SPARK_LIFE
+       pygame.draw.circle(screen, (255, int(160 + 95 * fade), int(60 * fade)), (int(pos.x), int(pos.y)), max(1, int(3 * fade)))
    if self.bullet:

```

```
pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1, self.bullet.y - 6, 3, 10))
if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:
```

Commit: Task 3: on_segment_hit spawns a spark burst where the segment was shot

Task 4: wave_speed_bonus

Returns $1 + 0.15 * (\text{wave} - 1)$. update divides TICK by this value, so each later wave steps segments more often: 1.0x on wave 1, 1.15x on wave 2, 1.6x on wave 5.

```
diff --git a/game.py b/game.py
index c3b09cc..f54a21c 100644
--- a/game.py
+++ b/game.py
@@ -29,7 +29,7 @@ def on_segment_hit(segment, score):

    def wave_speed_bonus(wave):
        """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default speed."""
-       pass
+       return 1 + 0.15 * (wave - 1)

    class Segment:
```

Commit: Task 4: wave_speed_bonus speeds the centipede up 15% per wave

Headless verification (SDL_VIDEODRIVER=dummy)

```
Task 1/2 hit 1: alive=True, flashing=True
Task 1/2 hit 2: alive=True, flashing=True
Task 1/2 hit 3: alive=True, flashing=True
Task 1/2 hit 4: alive=False, flashing=False
Task 2 colour when flashing: (255, 255, 255) | not flashing: None
Task 2 flash cleared after 0.11 s: True
Task 3 hit segment 4 of 11: chains [4, 6], sparks 12, score 10
Task 3 sparks left after 0.5 s: 0
Task 4 multipliers waves 1-5: [1.0, 1.15, 1.3, 1.45, 1.6]
All 120 frames of update + draw ran without errors
```

Final game.py

```
import random
import pygame

CELL, COLS, ROWS = 20, 30, 30
WIDTH, HEIGHT = COLS * CELL, ROWS * CELL + 30
ZONE_TOP = ROWS - 6
TICK, PLAYER_SPEED, BULLET_SPEED = 0.09, 260, 620
MUSHROOM_HP = 4
FLASH_TIME = 0.1
SPARK_LIFE = 0.4
sparks = []

def mushroom_color(hp, flashing=False):
    """Return an (r, g, b) colour for a mushroom with the given hit points, or None for the default."""
    if flashing:
        return (255, 255, 255)
    return None

def on_segment_hit(segment, score):
    """Called whenever a centipede segment is shot; add sparkles, sounds, or bonus points here."""
    cx = segment.col * CELL + CELL / 2
    cy = segment.row * CELL + CELL / 2
    for _ in range(12):
        velocity = pygame.Vector2(random.uniform(80, 220), 0).rotate(random.uniform(0, 360))
        sparks.append([pygame.Vector2(cx, cy), velocity, SPARK_LIFE])

def wave_speed_bonus(wave):
    """Return an extra tick-rate multiplier for centipede segments at the given wave, or None for the default speed."""
    return 1 + 0.15 * (wave - 1)

class Segment:
    def __init__(self, row, col, direction):
        self.row, self.col, self.dx, self.dy = row, col, direction, 1

    def step(self, mushrooms):
        nxt = self.col + self.dx
        if nxt < 0 or nxt >= COLS or (self.row, nxt) in mushrooms:
            self.row += self.dy
            self.dx = -self.dx
            if self.row >= ROWS - 1:
                self.row, self.dy = ROWS - 1, -1
            elif self.row <= ZONE_TOP and self.dy < 0:
```

```

        self.dy = 1
    else:
        self.col = nxt

class Game:
    def __init__(self):
        self.font = pygame.font.Font(None, 26)
        self.reset()

    def reset(self):
        self.score, self.lives, self.wave, self.state = 0, 3, 1, "play"
        self.mushrooms = {}
        self.flash = {}
        sparks.clear()
        for _ in range(45):
            self.mushrooms[(random.randint(1, ZONE_TOP - 2), random.randint(0, COLS - 1))] = MUSHROOM_HP
        self.respawn()
        self.spawn_wave()

    def respawn(self):
        self.x = WIDTH / 2
        self.y = HEIGHT - 50
        self.bullet = None
        self.invulnerable = 1.5

    def spawn_wave(self):
        length = 10 + self.wave
        self.chains = [[Segment(0, COLS - 1 - i, -1) for i in range(length)]]
        self.timer = 0.0

    def fire(self):
        if self.bullet is None and self.state == "play":
            self.bullet = pygame.Vector2(self.x, self.y - 12)

    def hit_mushroom(self, cell):
        self.mushrooms[cell] -= 1
        if self.mushrooms[cell] <= 0:
            del self.mushrooms[cell]
            self.flash.pop(cell, None)
            self.score += 5
        else:
            self.flash[cell] = FLASH_TIME

    def split_chain(self, chain, index):
        segment = chain[index]
        self.chains.remove(chain)
        left, right = chain[:index], chain[index + 1:]
        self.chains.extend(part for part in (left, right) if part)
        self.mushrooms[(segment.row, segment.col)] = MUSHROOM_HP
        self.score += 100 if index == 0 else 10
        on_segment_hit(segment, self.score)

    def update_bullet(self, dt):
        if self.bullet is None:
            return
        self.bullet.y -= BULLET_SPEED * dt
        cell = (int(self.bullet.y // CELL), int(self.bullet.x // CELL))
        if self.bullet.y < 0:
            self.bullet = None
        elif cell in self.mushrooms:
            self.hit_mushroom(cell)
            self.bullet = None
        else:
            for chain in self.chains:
                for index, segment in enumerate(chain):
                    if (segment.row, segment.col) == cell:
                        self.split_chain(chain, index)
                        self.bullet = None
                        return

    def update(self, dt, keys):
        if self.state != "play":
            return
        self.invulnerable = max(0.0, self.invulnerable - dt)
        self.flash = {cell: t - dt for cell, t in self.flash.items() if t - dt > 0}
        for spark in sparks:
            spark[0] += spark[1] * dt
            spark[2] -= dt
        sparks[:] = [spark for spark in sparks if spark[2] > 0]
        self.x += (keys[pygame.K_RIGHT] - keys[pygame.K_LEFT]) * PLAYER_SPEED * dt
        self.y += (keys[pygame.K_DOWN] - keys[pygame.K_UP]) * PLAYER_SPEED * dt
        self.x = max(10, min(WIDTH - 10, self.x))
        self.y = max(ZONE_TOP * CELL + 10, min(ROWS * CELL - 10, self.y))
        if keys[pygame.K_SPACE]:
            self.fire()
        self.update_bullet(dt)
        self.timer += dt
        effective_tick = TICK / (wave_speed_bonus(self.wave) or 1)
        while self.timer >= effective_tick:
            self.timer -= effective_tick
            for chain in self.chains:

```

```

        for segment in chain:
            segment.step(self.mushrooms)
        player_cell = (int(self.y // CELL), int(self.x // CELL))
        if self.invulnerable <= 0 and any((s.row, s.col) == player_cell for c in self.chains for s in c):
            self.lives -= 1
            self.respawn()
            if self.lives <= 0:
                self.state = "lose"
        if not self.chains:
            self.wave += 1
            self.spawn_wave()

def draw(self, screen):
    screen.fill((8, 8, 16))
    for (row, col), hp in self.mushrooms.items():
        color = mushroom_color(hp, (row, col) in self.flash) or (200 - (MUSHROOM_HP - hp) * 40, 80, 170)
        center = (col * CELL + CELL // 2, row * CELL + CELL // 2)
        pygame.draw.circle(screen, color, center, CELL // 2 - 1)
        pygame.draw.rect(screen, (230, 230, 200), (center[0] - 3, center[1], 6, CELL // 2 - 1))
    for chain in self.chains:
        for index, segment in enumerate(chain):
            center = (segment.col * CELL + CELL // 2, segment.row * CELL + CELL // 2)
            pygame.draw.circle(screen, (240, 200, 60) if index == 0 else (80, 220, 90), center, CELL // 2)
    for pos, _, life in sparks:
        fade = life / SPARK_LIFE
        pygame.draw.circle(screen, (255, int(160 + 95 * fade), int(60 * fade)), (int(pos.x), int(pos.y)), max(1, int(3 * fade)))
    if self.bullet:
        pygame.draw.rect(screen, (255, 255, 255), (self.bullet.x - 1, self.bullet.y - 6, 3, 10))
    if self.invulnerable <= 0 or int(self.invulnerable * 10) % 2 == 0:
        pygame.draw.polygon(screen, (80, 200, 255), [(self.x, self.y - 10), (self.x + 10, self.y + 10), (self.x - 10, self.y + 10)])
    hud = self.font.render(f"Score {self.score} Lives {self.lives} Wave {self.wave} R = reset", True, (240, 240, 240))
    screen.blit(hud, (10, ROWS * CELL + 6))
    if self.state == "lose":
        label = self.font.render("GAME OVER - Press R", True, (255, 255, 120))
        screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 2)))

def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Centipede")
    clock = pygame.time.Clock()
    game = Game()
    running = True
    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                game.reset()
        game.update(dt, pygame.key.get_pressed())
        game.draw(screen)
        pygame.display.flip()
    pygame.quit()

if __name__ == "__main__":
    main()

```